💻 Cahier d'Exercices : Pratique du C++ et du C#

Bienvenue dans ton espace d'entraînement ! La programmation s'apprend avant tout avec les doigts sur le clavier.

Pour chaque module du guide, tu trouveras ici des exercices progressifs. **L'objectif :** réaliser chaque exercice en **C++** ET en **C#**, et obtenir exactement le même résultat dans ton terminal que celui affiché dans les blocs "Sortie attendue".

Note : Dans les sorties attendues, les textes précédés ou entourés d'une indication comme `[Saisie utilisateur : ...]` représentent ce que le joueur tape au clavier.

Module 1 : Le "Hello World" et la structure d'un programme

Exercice 1.1 : L'écran-titre

Écris un programme qui affiche l'écran-titre d'un jeu vidéo fictif sur trois lignes distinctes, entouré de bordures en texte.

Sortie terminale attendue :

```
==============================

  LA LEGENDE DU CODEUR

==============================
```

Exercice 1.2 : Le journal de bord

Affiche successivement trois messages qui racontent le réveil d'un héros. Ajoute au moins deux commentaires `//` dans ton code pour expliquer ce que font tes lignes.

Sortie terminale attendue :

```
Jour 1 : Je me reveille dans une foret sombre.

Je trouve une vieille epee rouillee par terre.

L'aventure commence maintenant !
```

Exercice 1.3 : Le dessin en texte (ASCII Art)

Utilise plusieurs instructions d'affichage pour dessiner une petite épée ou un bouclier dans la console. Fais bien attention aux espaces !

Sortie terminale attendue :

```
    /| ________________
O|===|* >________________>
    \|
```

Module 2 : Les Variables et les Interactions

Section 1 : Création et affichage de variables

Exercice 2.1 : La fiche d'inventaire

Crée 4 variables pour représenter un objet magique :

- Son nom (`string`) : `"Anneau de Vitesse"`
- Son niveau requis (`int`) : `5`
- Son poids en kilos (`double`) : `0.25`
- S'il est équipé ou non (`bool`) : `true`

Affiche ensuite toutes ces informations proprement. *(Note : en C++, un `bool` affiche `1` pour vrai et `0` pour faux par défaut, tandis qu'en C# il affiche `True` ou `False`. C'est tout à fait normal !)*

Sortie terminale attendue (Exemple C#) :

Objet : Anneau de Vitesse

Niveau requis : 5

Poids : 0.25 kg

Equipe : True

Exercice 2.2 : Le score évolutif

Crée une variable entière `score` initialisée à `0`. Affiche-la. Ajoute-lui `100` points, affiche-la. Multiplie-la par `2`, puis affiche le score final.

Sortie terminale attendue :

Score initial : 0

Monstre battu (+100) ! Score : 100

Bonus x2 active ! Score final : 200

Section 2 : Interactions avec l'utilisateur et Calculs

Exercice 2.3 : Le cri de guerre personnalisé

Demande à l'utilisateur le nom de son personnage ainsi que le nom de son arme favorite (deux variables `string`), puis affiche une phrase épique combinant les deux. *(Attention en C++ : tape des mots sans espaces avec `std::cin`, par exemple "Excalibur")*.

Sortie terminale attendue :

Quel est ton nom, aventurier ? Lancelot

Quelle est ton arme favorite ? Excalibur

Tremblez ennemis ! Lancelot arrive avec sa redoutable Excalibur !

Exercice 2.4 : Le marchand de potions

Une potion de soin coûte **15 pièces d'or**. Demande au joueur combien de potions il souhaite acheter (`int`), calcule le coût total et affiche la facture.

Sortie terminale attendue :

Bienvenue dans ma boutique ! Une potion coute 15 pieces d'or.

Combien de potions veux-tu acheter ? 4

Cela te coutera 60 pieces d'or. Merci et a bientot !

Exercice 2.5 : Le partage du butin

Demande au joueur combien de pièces d'or le groupe a trouvé dans le donjon (`int`), puis demande combien de joueurs composent l'équipe (`int`). Affiche combien de pièces chaque joueur recevra en divisant le butin.

Sortie terminale attendue :

Pieces d'or trouvees dans le coffre : 100

Nombre de joueurs dans le groupe : 4

Chaque joueur repart avec 25 pieces d'or !

Module 3 : Prendre des décisions (Les Conditions)

Section 1 : Le bloc `if / else if / else`

Exercice 3.1 : Le videur de la taverne

Demande le niveau (`int`) du joueur. S'il est supérieur ou égal à `10`, laisse-le entrer dans la taverne des vétérans. Sinon, refuse-lui l'accès.

Sortie terminale attendue (Cas 1 - Niveau 12) :

Halte la ! Quel est ton niveau ? 12

Entre donc, grand guerrier !

Sortie terminale attendue (Cas 2 - Niveau 4) :

Halte la ! Quel est ton niveau ? 4

Desole gamin, il faut etre au moins niveau 10 pour entrer ici.

Exercice 3.2 : Le radar de température

Demande la température actuelle de la zone (`int`) et affiche le climat correspondant :

- Moins de `0` : Terres gelées
- Entre `0` et `30` inclus : Climat tempéré

- Plus de `30` : Désert brûlant

Sortie terminale attendue (Exemple avec -15) :

Quelle est la temperature actuelle ? -15

Brrr ! Vous entrez dans les Terres gelees. Equipez un manteau !

Exercice 3.3 : Le coffre à double serrure (`&&` et `||`)

Pour ouvrir le coffre royal, le joueur doit avoir la clé (`bool aLaCle = true`) **ET** avoir un niveau de crochetage d'au moins 50 (`int crochetage`). Demande au joueur son niveau de crochetage et vérifie si les deux conditions sont réunies.

Sortie terminale attendue (Exemple avec 65) :

Vous possedez la cle en argent.

Quel est votre niveau de crochetage ? 65

Clic ! Les deux serrures s'ouvrent, le tresor est a vous !

Section 2 : Le `switch`

Exercice 3.4 : Le choix de la classe

Affiche une liste de 3 classes de départ. Demande au joueur de taper un chiffre entre 1 et 3 et utilise un `switch` pour afficher son équipement de départ (et un message d'erreur avec `default` s'il tape autre chose).

Sortie terminale attendue :

Choisissez votre classe :

1 - Guerrier

2 - Mage

3 - Voleur

Votre choix : 2

Vous recevez un Baton Magique et une Robe d'Apprenti !

Exercice 3.5 : Le téléporteur directionnel

Le joueur se trouve devant un téléporteur à 4 boutons : `1` (Nord), `2` (Sud), `3` (Est), `4` (Ouest). Utilise un `switch` pour lui indiquer dans quelle région il atterrit selon son choix.

Sortie terminale attendue :

Dans quelle direction activer le teleporteur (1:Nord, 2:Sud, 3:Est, 4:Ouest) ? 9

Erreur : Coordonnees inconnues, le teleporteur grésille mais rien ne se passe.

Module 4 : Répéter des actions (Les Boucles)

Section 1 : La boucle `while`

Exercice 4.1 : Le compte à rebours explosif

Crée une variable `secondes = 5`. Utilise une boucle `while` pour afficher le décompte jusqu'à 1, puis affiche `"BOUM !"` après la boucle.

Sortie terminale attendue :

Bombe amorcee !

5...

4...

3...

2...

1...

BOUM !

Exercice 4.2 : La méditation du Mage

Un mage commence avec `20` points de mana. Son mana maximum est de `100`. Tant que son mana est inférieur à `100`, il médite et récupère `20` points de mana par tour. Affiche l'évolution de son mana à chaque étape.

Sortie terminale attendue :

Debut de la meditation (Mana : 20/100)

Meditation en cours... Mana : 40/100

Meditation en cours... Mana : 60/100

Meditation en cours... Mana : 80/100

Meditation en cours... Mana : 100/100

Mana au maximum ! Pret au combat.

Section 2 : La boucle `do...while`

Exercice 4.3 : L'énigme du Sphinx

Le Sphinx demande : *"Combien de pattes possède une araignée géante ?"*. Utilise une boucle `do...while` pour reposer la question en boucle tant que l'utilisateur ne répond pas `8`.

Sortie terminale attendue :

Sphinx : Combien de pattes possede une araignee geante ? 6

Sphinx : Combien de pattes possede une araignee geante ? 10

Sphinx : Combien de pattes possede une araignee geante ? 8

Sphinx : Bonne reponse ! Tu peux franchir la porte.

Exercice 4.4 : Validation de jauge

Demande à l'utilisateur de choisir une difficulté comprise **entre 1 et 5 inclus**. Tant que le nombre saisi est inférieur à 1 OU supérieur à 5, redemande-lui de saisir une valeur valide.

Sortie terminale attendue :

Choisissez la difficulte du jeu (entre 1 et 5) : 0

Choisissez la difficulte du jeu (entre 1 et 5) : 9

Choisissez la difficulte du jeu (entre 1 et 5) : 3

Difficulte reglee sur le niveau 3. Bon jeu !

Section 3 : La boucle `for`

Exercice 4.5 : L'entraînement aux pompes

Le héros doit faire 6 pompes pour gagner en force. Utilise une boucle `for` allant de 1 à 6 pour afficher chaque répétition.

Sortie terminale attendue :

Debut de l'entrainement :

Pompe numero 1 !

Pompe numero 2 !

Pompe numero 3 !

Pompe numero 4 !

Pompe numero 5 !

Pompe numero 6 !

Force +1 !

Exercice 4.6 : La table des multiplicateurs de dégâts

Un sort inflige 7 points de dégâts de base par niveau de charge. Utilise une boucle `for` de 1 à 5 pour afficher les dégâts du sort aux niveaux de charge 1, 2, 3, 4 et 5 (c'est-à-dire la table de multiplication de 7).

Sortie terminale attendue :

Charge niveau 1 : 7 degats

Charge niveau 2 : 14 degats

Charge niveau 3 : 21 degats

Charge niveau 4 : 28 degats

Charge niveau 5 : 35 degats

Exercice 4.7 : Les vagues paires (Bonus pas de 2)

Modifie l'étape d'augmentation de ta boucle `for` (utilise `i += 2` au lieu de `i++`) pour afficher uniquement les numéros pairs de 2 à 10.

Sortie terminale attendue :

Vague ennemie n°2

Vague ennemie n°4

Vague ennemie n°6

Vague ennemie n°8

Vague ennemie n°10

Section 4 : Les Tableaux et la boucle `foreach`

Exercice 4.8 : Le grimoire de sorts

Crée un tableau de 4 textes (`string`) contenant les sorts : `"Boule de feu"`, `"Eclair"`, `"Soin"`, `"Bouclier de glace"`. Parcours-le avec une boucle `foreach` (ou `for(:)` en C++) pour tous les afficher.

Sortie terminale attendue :

Sorts inscrits dans le grimoire :

* Boule de feu

* Eclair

* Soin

* Bouclier de glace

Exercice 4.9 : Le total du trésor

Crée un tableau de 5 nombres entiers (`int`) représentant la valeur de 5 pierres précieuses trouvées : `10`, `50`, `25`, `100`, `5`. Crée une variable `somme = 0` avant la boucle, puis parcours le tableau pour additionner chaque pierre à la somme totale.

Sortie terminale attendue :

Ajout d'une pierre de 10 PO...

Ajout d'une pierre de 50 PO...

Ajout d'une pierre de 25 PO...

Ajout d'une pierre de 100 PO...

Ajout d'une pierre de 5 PO...

Valeur totale du tresor : 190 PO !

Module 5 : Les Fonctions (Sous-programmes)

Exercice 5.1 : Le héraut du roi (Fonction `void` avec paramètre)

Crée une fonction `saluerJoueur` qui ne renvoie rien (`void`), mais qui prend en paramètre un `string nom` et un `int niveau`. Appelle cette fonction deux fois dans ton `main` avec des personnages différents.

Sortie terminale attendue :

Oyez, oyez ! Saluez tous Aragorn, glorieux heros de niveau 20 !

Oyez, oyez ! Saluez tous Frodon, glorieux heros de niveau 3 !

Exercice 5.2 : Le forgeron nain (Fonction avec `return` entier)

Crée une fonction `calculerPrixReparation` qui prend en paramètre un `int pointsDurabilitePerdus`, multiplie ce nombre par `3`, et renvoie (`return`) le résultat. Récupère et affiche ce prix dans ton `main`.

Sortie terminale attendue :

Votre armure a perdu 15 points de durabilite.

Le forgeron vous demande 45 pieces d'or pour la reparer.

Exercice 5.3 : Vérificateur de survie (Fonction avec `return` booléen)

Crée une fonction `estEnVie` qui prend en paramètre un `int pointsDeVie` et renvoie `true` si les PV sont strictement supérieurs à `0`, et `false` sinon. Utilise cette fonction directement dans un `if` à l'intérieur de ton `main`.

Sortie terminale attendue :

Test avec 45 PV : Le personnage est toujours debout !

Test avec 0 PV : Le personnage est K.O...

Module 6 : La Programmation Orientée Objet (POO)

Section 1 & 2 : Classes, Objets, Constructeur et Encapsulation

Exercice 6.1 : L'Armurerie

Crée une classe `Arme` avec :

- Deux attributs privés (`private`) : `nom` (`string`) et `degats` (`int`).
- Un constructeur public pour initialiser ces deux valeurs.
- Une méthode publique `afficherInfos()` qui affiche ses caractéristiques.

Dans le `main`, instancie deux armes différentes et affiche leurs informations.

Sortie terminale attendue :

Arme : Dague en fer | Degats : 8

Arme : Marteau de guerre | Degats : 32

Exercice 6.2 : La Bourse d'Or sécurisée

Crée une classe `Bourse` avec un attribut privé `piecesOr` (`int`), initialisé à `0` dans le constructeur (sans paramètre). Ajoute trois méthodes publiques :

- `ajouterOr(int montant)` : ajoute le montant à la bourse.
- `depenserOr(int montant)` : vérifie s'il y a assez d'or. Si oui, retire le montant. Si non, affiche un message d'erreur sans modifier la bourse !
- `afficherSolde()` : affiche l'or disponible.

Sortie terminale attendue :

Solde actuel : 0 PO

+50 PO ajoutes a la bourse.

Solde actuel : 50 PO

Achat d'une epee a 30 PO : Succes !

Solde actuel : 20 PO

Tentative d'achat d'un chateau a 1000 PO : Fonds insuffisants ! Il vous manque de l'or.

Solde actuel : 20 PO

Exercice 6.3 : La fontaine de guérison (Interaction entre objets)

En reprenant le modèle de la classe `Personnage` du cours (avec un nom, des `pv` et des `pvMax`), ajoute une méthode `soigner(Personnage& cible, int pointsDeSoin)` (en C++) / `Soigner(Personnage cible, int pointsDeSoin)` (en C#). Assure-toi que les PV de la cible ne dépassent jamais ses `pvMax` !

Sortie terminale attendue :

[Paladin] PV : 100/100

[Guerrier blesse] PV : 40/100

Paladin lance un sort de soin sur Guerrier blesse (+35 PV) !

[Guerrier blesse] PV : 75/100

Paladin lance un sort de soin sur Guerrier blesse (+35 PV) !

[Guerrier blesse] PV : 100/100

Section 3 : L'Héritage

Exercice 6.4 : Le Bestiaire (Parent et Enfants)

1. Crée une classe parente `Monstre` avec des attributs `protected` (`nom` et `pv`), un constructeur et une méthode `crier()` qui affiche `"Grrr !"`.
2. Crée une classe `Slime` qui hérite de `Monstre` et possède une méthode `seDiviser()`.
3. Crée une classe `Dragon` qui hérite de `Monstre`, possède un attribut supplémentaire `porteeFlammes` (`int`) et une méthode `cracherDuFeu()`.

Teste tes deux monstres dans le `main`.

Sortie terminale attendue :

--- Apparition d'un Slime (20 PV) ---

Slime : Grrr !

Slime se divise en deux petits Slimes gluants !

--- Apparition d'un Dragon Rouge (500 PV) ---

Dragon Rouge : Grrr !

Dragon Rouge crache des flammes sur 30 metres !

🏆 Module 7 : Défis d'évolution sur le Projet Final ("Les Chroniques de l'Arène")

Une fois que tu as compilé et joué au jeu de base du Module 7, modifie son code pour relever ces 3 défis !

Défi 7.1 : La Posture Défensive (Nouveau choix dans le menu)

Ajoute une 3ème option dans le menu du joueur : `3: Se defendre`. Lorsque le joueur choisit cette action, il inflige de légers dégâts au bouclier à l'ennemi (`5` points de dégâts fixes) et se soigne simultanément de `10` PV grâce à son souffle repris. N'oublie pas de mettre à jour la condition de ta boucle `do...while` pour accepter le chiffre 3 !

Sortie terminale attendue (Extrait d'un tour) :

Que veux-tu faire ? (1: Attaquer | 2: Boire une potion | 3: Se defendre) : 3

🛡️ Arthur leve son bouclier, repousse Gobelin Eclaireur et reprend son souffle (+10 PV) !

-> Gobelin Eclaireur subit 5 degats ! (PV restants : 40)

Défi 7.2 : Le Butin de Victoire (Loot)

Lorsqu'un ennemi est vaincu (`!ennemiActuel.estVivant()`), récompense le héros en lui faisant trouver automatiquement **1 nouvelle potion** sur le corps du monstre et en augmentant sa **Force de +5** pour les combats suivants (crée une méthode `gagnerRecompense()` dans `Combattant`).

Sortie terminale attendue (Extrait de fin de combat) :

🏆 Victoire ! Gobelin Eclaireur a ete terrasse !

🎁 Butin : Vous trouvez 1 potion et votre Force augmente de +5 !

Défi 7.3 : L'Attaque Lourde risquée

Ajoute un attribut `mana` au `Combattant` (le héros commence avec `30` points de mana). Ajoute un choix `4: Attaque Lourde (15 Mana)` dans le menu. Si le héros a assez de mana, il consomme 15 points de mana et inflige **le double de sa force** (`force * 2`) à l'ennemi !

Sortie terminale attendue (Extrait d'un tour) :

Que veux-tu faire ? (1: Attaquer | 2: Potion | 3: Defendre | 4: Attaque Lourde) : 4

⚡ Arthur concentre son energie (-15 Mana) et declenche une ATTAQUE LOURDE sur Dragon Noir (BOSS) !

-> Dragon Noir (BOSS) subit 50 degats ! (PV restants : 60)